**Assignment 4 — BigQuery Query Optimization with Partitioning and Clustering**

**Case Study: NYC Taxi Trip Analytics Performance Challenge**

**A transport analytics team has started analysing New York City Yellow Taxi trip data in BigQuery.**

**The team first loaded all records into a normal BigQuery table. Analysts frequently filter by pickup date and taxi-zone locations. As the table grows, the team wants to reduce unnecessary data scanning and improve query efficiency.**

**Your task is to load a realistic public dataset, establish a baseline, and redesign the table using BigQuery partitioning and clustering.**

**Public Data Source**

**Use the official NYC Taxi & Limousine Commission Yellow Taxi Trip Record files.**

**January 2024**

**https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet**

**February 2024**

**https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-02.parquet**

**March 2024**

**https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-03.parquet**

**Official source page:**

**https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page**

**Recommended BigQuery Dataset**

**taxi_optimization**

**Important Fields to Investigate**

**tpep_pickup_datetime**

**tpep_dropoff_datetime**

**passenger_count**

**trip_distance**

**PULocationID**

**DOLocationID**

**payment_type**

**fare_amount**

**total_amount**

**Business Questions**

**Management regularly asks:**

- **How many trips occurred in a selected date range?**
- **What revenue came from a particular pickup location?**
- **Which pickup/drop-off combinations generated the most trips?**
- **How do results differ by payment type?**
- **What is the average trip distance for a selected week and pickup zone?**

**The initial table is not partitioned or clustered.**

**Assignment Requirements**

**Phase 1 — Load the Data**

- **Download the three public Parquet files.**
- **Store them in a location from which BigQuery can load them reliably.**
- **Load all three months into one normal BigQuery native table.**
- **Validate row count and pickup date range.**

**Phase 2 — Baseline**

**Create at least three business queries using combinations of:**

**pickup date/date range**

**pickup location**

**drop-off location**

**payment type**

**For each query capture:**

- **Estimated bytes processed where available.**
- **Actual bytes processed from job details.**
- **Result returned.**
- **Comparable execution information.**

**Do not compare only elapsed time because result caching can influence repeated-query timing.**

**Phase 3 — Choose Optimization**

**Decide and justify:**

- **Partition field.**
- **Partition granularity.**
- **One to three clustering fields.**
- **Clustering order.**

**The design must be based on the business filter patterns.**

**Phase 4 — Create Optimized Table**

**Create a second BigQuery table with the same relevant data using your chosen partition and clustering design.**

**Phase 5 — Compare**

**Run the same business questions against both the normal and optimized tables.**

**Explain:**

- **Partition pruning.**
- **Cluster/block pruning.**
- **Why date filters can reduce scanning.**
- **Why cluster keys should match common selective filters.**
- **Why cluster column order matters.**

**Required Comparison**

**Create a result table similar to:**

| Test | Normal Table | Optimized Table |
|---|---|---|
| Query 1 bytes processed | ... | ... |
| Query 2 bytes processed | ... | ... |
| Query 3 bytes processed | ... | ... |

**Do not invent byte values. Use your own BigQuery job results.**

**Investigation Questions**

- **Why is pickup timestamp a strong partition candidate?**
- **Why would payment type alone be a poor partition key for this workload?**
- **Why can a date filter reduce scanned partitions?**
- **What advantage can location-based clustering provide?**
- **Does clustering guarantee every query is faster?**
- **Why should frequently used/selective clustering fields appear earlier?**
- **Why should table optimization be based on query patterns?**

**Deliverables**

**Submit source evidence, BigQuery screenshots, raw-table validation, three baseline results, optimized-table details, the same three optimized-query results, a bytes-processed comparison, partition/cluster justification, and a short conclusion.**

**Assignment 4 — BigQuery Query Optimization with Partitioning and Clustering**

**Case Study: NYC Taxi Trip Analytics Performance Challenge**

A transport analytics team has started analysing New York City Yellow Taxi trip data in BigQuery.

The team first loaded all records into a normal BigQuery table. Analysts frequently filter by pickup date and taxi-zone locations. As the table grows, the team wants to reduce unnecessary data scanning and improve query efficiency.

Your task is to load a realistic public dataset, establish a baseline, and redesign the table using BigQuery partitioning and clustering.

**Public Data Source**

Use the official NYC Taxi & Limousine Commission Yellow Taxi Trip Record files.

**January 2024**

https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet

**February 2024**

https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-02.parquet

**March 2024**

https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-03.parquet

Official source page:

https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

**Recommended BigQuery Dataset**

taxi_optimization

**Important Fields to Investigate**

tpep_pickup_datetime

tpep_dropoff_datetime

passenger_count

trip_distance

PULocationID

DOLocationID

payment_type

fare_amount

total_amount

**Business Questions**

Management regularly asks:

- How many trips occurred in a selected date range?
- What revenue came from a particular pickup location?
- Which pickup/drop-off combinations generated the most trips?
- How do results differ by payment type?
- What is the average trip distance for a selected week and pickup zone?

The initial table is not partitioned or clustered.

**Assignment Requirements**

**Phase 1 — Load the Data**

- Download the three public Parquet files.
- Store them in a location from which BigQuery can load them reliably.
- Load all three months into one normal BigQuery native table.
- Validate row count and pickup date range.

**Phase 2 — Baseline**

Create at least three business queries using combinations of:

pickup date/date range

pickup location

drop-off location

payment type

For each query capture:

- Estimated bytes processed where available.
- Actual bytes processed from job details.
- Result returned.
- Comparable execution information.

Do not compare only elapsed time because result caching can influence repeated-query timing.

**Phase 3 — Choose Optimization**

Decide and justify:

- Partition field.
- Partition granularity.
- One to three clustering fields.
- Clustering order.

The design must be based on the business filter patterns.

**Phase 4 — Create Optimized Table**

Create a second BigQuery table with the same relevant data using your chosen partition and clustering design.

**Phase 5 — Compare**

Run the same business questions against both the normal and optimized tables.

Explain:

- Partition pruning.
- Cluster/block pruning.
- Why date filters can reduce scanning.
- Why cluster keys should match common selective filters.
- Why cluster column order matters.

**Required Comparison**

Create a result table similar to:

| Test | Normal Table | Optimized Table |
|---|---|---|
| Query 1 bytes processed | ... | ... |
| Query 2 bytes processed | ... | ... |
| Query 3 bytes processed | ... | ... |

Do not invent byte values. Use your own BigQuery job results.

**Investigation Questions**

- Why is pickup timestamp a strong partition candidate?
- Why would payment type alone be a poor partition key for this workload?
- Why can a date filter reduce scanned partitions?
- What advantage can location-based clustering provide?
- Does clustering guarantee every query is faster?
- Why should frequently used/selective clustering fields appear earlier?
- Why should table optimization be based on query patterns?

**Deliverables**

Submit source evidence, BigQuery screenshots, raw-table validation, three baseline results, optimized-table details, the same three optimized-query results, a bytes-processed comparison, partition/cluster justification, and a short conclusion.